

MSX ONLINE

GAME SERVER

Documento de Arquitectura y Protocolo

Versión 1.0 · Stack: Node.js + TCP raw · Protocolo: UNAPI TCP/IP · Hardware MSX: ObsoNET + InterNestor Lite
MSXgl · unapi_tcp.h · Ubuntu Server 24.04 LTS · VPS con IP pública

COMPONENTE	TECNOLOGÍA	ROL
MSX (cliente)	ObsoNET + InterNestor Lite MSXgl / unapi_tcp.h	Envía y recibe estado de juego
Protocolo	TCP raw · Binario propio Header 6 bytes fijos	Relay de paquetes entre jugadores
Servidor	Node.js 20 · net.createServer() Ubuntu 24.04 minimal VPS	Gestión de salas y reenvío
Seguridad	Token auth 4 bytes ufw firewall + fail2ban	Sin TLS (inviabile en Z80)

Este documento recoge la arquitectura, el protocolo de comunicación y el código base del servidor de partidas online para MSX. El servidor actúa como relay agnóstico al juego: gestiona salas de hasta 4 jugadores y reenvía paquetes de estado sin interpretar su contenido, permitiendo conectar cualquier juego MSX sin modificar el servidor.

1. ARQUITECTURA GENERAL

El sistema sigue un modelo **star topology**: todos los MSX conectan al servidor central. El servidor no implementa lógica de juego — solo sabe de *rooms*, *players* y *relay de paquetes*. Añadir un nuevo juego es cero cambios en el servidor.

```
[ MSX Player 1 ] ████
[ MSX Player 2 ] ████████ TCP / Internet ███ [ UBUNTU VPS ]
[ MSX Player 3 ] ██████ Node.js Game Server
[ MSX Player 4 ] ██████ Salas · Relay · Auth

Hardware MSX: ObsoNET + InterNestor Lite (TCP/IP UNAPI 1.1)
Librería MSX: MSXgl · unapi_tcp.h

Flujo de un tick de juego:

P1 mueve sprite
  → STATE_UPDATE → Servidor
    → relay a P2, P3, P4 de la misma sala
      → cada MSX actualiza pantalla
```

1.1 Principios de diseño

- El servidor es **agnóstico al juego**: no interpreta el payload de STATE_UPDATE.
- Añadir un nuevo juego = definir el payload en el MSX. El servidor no cambia.
- El servidor NO implementa lógica de validación de movimientos ni reglas.
- El protocolo es **request/response secuencial** — no hay multihilo en Z80.
- Máximo **4 jugadores por sala** · hasta **255 salas simultáneas**.

1.2 Responsabilidades claras

EL SERVIDOR SABE / HACE	EL SERVIDOR NO SABE / NO HACE
Quién está conectado	Qué significan X, Y en el payload
En qué sala está cada jugador	Reglas del juego
A quién reenviar cada paquete	Si un movimiento es válido
Cuántos jugadores hay en sala	Qué juego se está jugando (solo GAME_ID)
Gestionar desconexiones / drops	Sincronización de frames
Autenticar con token de 4 bytes	Cifrado o TLS (inviabile en Z80)

2. LIMITACIONES ObsoNET + InterNestor Lite

Estas restricciones son las que dictan las decisiones de diseño del protocolo. Ignorarlas causa bugs difíciles de reproducir y depurar.

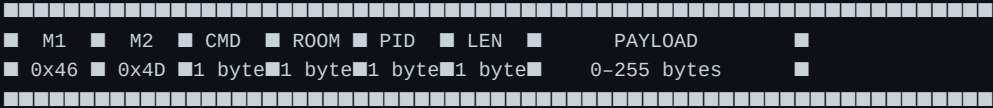
■ *InterNestor Lite procesa TCP en el interrupt del timer de 50 Hz. El servidor NUNCA debe enviar datos en ráfaga sin dar tiempo al Z80 a procesar.*

FACTOR	VALOR REAL	CONSECUENCIA EN EL DISEÑO
CPU MSX	~3.58 MHz, Z80, sin FPU	Timeouts generosos (90s). Payload lo más simple posible.
InterNestor timer	50 Hz = cada ~20 ms	No enviar en ráfaga. El servidor espera ACK antes del siguiente push.
Buffer TCP	~512–1024 bytes típico	Payload máximo 255 bytes por paquete.
Sin TLS	Imposible en Z80	Auth por token en el protocolo. Canal en claro.
ObsoNET estabilidad	Drops frecuentes posibles	El servidor tolera ECONNRESET sin error grave. Reconexión limpia.
Chunks TCP	Paquete puede llegar en varios frames	Soporte de acumulación obligatorio en el servidor Node.

3. PROTOCOLO BINARIO

El protocolo está diseñado para ser parseable con aritmética básica en Z80. Header fijo de **6 bytes**. Payload máximo de **255 bytes**. Sin campos de longitud de 16 bits (requieren dos fetches y ensamblaje en Z80).

3.1 Estructura del paquete



Magic: 0x46 ('F') + 0x4D ('M') → FM de FX-Media
CMD: Comando de aplicación (1 byte = 256 comandos posibles)
ROOM: ID de sala 0x00 = sin sala | 0x01-0xFF = sala activa
PID: Player ID 0x00 = servidor | 0x01-0x04 = jugadores
LEN: Longitud del payload (0 = sin payload)
PAYLOAD: Datos de aplicación (definidos por cada juego en STATE_UPDATE)

Total header: 6 bytes fijos. Tamaño máximo de paquete: 261 bytes.

3.2 Tabla de comandos

CONEXIÓN Y AUTENTICACIÓN

CMD	HEX	DIRECCIÓN	PAYLOAD	DESCRIPCIÓN
PING	0x01	MSX → SRV	—	Keepalive. Mantiene viva la conexión.
PONG	0x02	SRV → MSX	—	Respuesta al PING.
AUTH	0x10	MSX → SRV	Token 4 bytes	Autenticación. Primer paquete obligatorio.
AUTH_OK	0x11	SRV → MSX	—	Autenticado correctamente.
AUTH_FAIL	0x12	SRV → MSX	—	Token incorrecto → servidor cierra conexión.

GESTIÓN DE SALAS

CMD	HEX	DIRECCIÓN	PAYLOAD	DESCRIPCIÓN
ROOM_CREATE	0x20	MSX → SRV	GAME_ID (1B)	Crea sala nueva. El creador es P1 (host).
ROOM_JOIN	0x21	MSX → SRV	ROOM_ID (1B)	Unirse a sala existente.
ROOM_LEAVE	0x22	MSX → SRV	—	Abandonar sala actual.
ROOM_INFO	0x23	SRV → MSX	ROOM_ID+GAME_ID +N_PLAYERS+PID	Confirmación de sala. Incluye PID asignado.
ROOM_FULL	0x24	SRV → MSX	—	Sala llena (4/4 jugadores).
ROOM_NOT_FOUND	0x25	SRV → MSX	—	La sala solicitada no existe.

EVENTOS DE SALA (broadcasts del servidor)

CMD	HEX	DIRECCIÓN	PAYLOAD	DESCRIPCIÓN
PLAYER_JOINED	0x30	SRV → TODOS	PID (1B)	Avisa a todos cuando entra un jugador nuevo.
PLAYER_LEFT	0x31	SRV → TODOS	PID (1B)	Avisa a todos cuando un jugador abandona.
GAME_START	0x32	SRV → TODOS	—	El host (P1) ha iniciado la partida.
GAME_END	0x33	SRV → TODOS	—	Partida terminada.

DATOS DE JUEGO

CMD	HEX	DIRECCIÓN	PAYLOAD	DESCRIPCIÓN
STATE_UPDATE	0x40	MSX→SRV→MSX	Libre (0–255B)	EL CORAZÓN DEL SISTEMA. El servidor hace relay del paquete completo
ERROR	0xFF	Ambos	Código (1B)	Error genérico.

4. FLUJO DE SESIÓN COMPLETO

4.1 Conexión, autenticación y creación/unión a sala



4.2 Partida en curso — relay de STATE_UPDATE



■ El campo PID en el header del paquete identifica al remitente. No es necesario incluir el PID en el payload. El MSX receptor lee el header y sabe de quién son los datos.

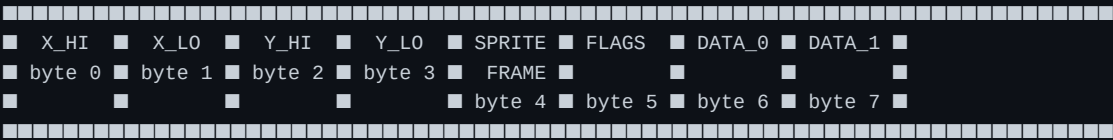
4.3 Desconexión de un jugador



5. PAYLOAD DE STATE_UPDATE — PROPUESTA BASE

El payload de STATE_UPDATE es **completamente libre** — cada juego define el suyo. El servidor lo retransmite sin tocarlo. A continuación se propone una estructura base de **8 bytes** válida para la mayoría de juegos de sprites 2D en MSX.

STATE_UPDATE payload – estructura base (8 bytes)



X: 16 bits → coordenada horizontal (0-511 píxeles)
Y: 16 bits → coordenada vertical (0-211 píxeles)
FRAME: 8 bits → frame de animación actual del sprite
FLAGS: 8 bits → bits de estado (ver tabla abajo)
DATA: 2 bytes → datos extra del juego (vida, puntuación, arma...)

Byte FLAGS (sugerido):

BIT	MÁSCARA	SIGNIFICADO
0	0x01	Dirección: 0=izquierda, 1=derecha
1	0x02	Dirección vertical: 0=arriba, 1=abajo
2	0x04	Acción A pulsada (disparo, salto...)
3	0x08	Acción B pulsada
4	0x10	En suelo / plataforma
5	0x20	Invencible / parpadeando
6	0x40	Libre para el juego
7	0x80	Libre para el juego

■ Para juegos que necesiten más datos (ej. un shooter con múltiples proyectiles), se puede enviar múltiples STATE_UPDATE seguidos, cada uno con LEN diferente. El servidor los retransmite todos en orden.

Ejemplo en C (MSXgl)

// Construcción del paquete STATE_UPDATE en MSX (C con MSXgl)

```
typedef struct {
    u8 x_hi, x_lo;    // Coordenada X
    u8 y_hi, y_lo;    // Coordenada Y
    u8 frame;         // Frame sprite
    u8 flags;         // Estado
    u8 data0, data1;  // Extra del juego
} StatePayload;

u8 packet[14];        // 6 header + 8 payload
StatePayload* pl = (StatePayload*)(packet + 6);

packet[0] = 0x46;      // Magic FM
packet[1] = 0x4D;
packet[2] = 0x40;      // CMD_STATE_UPDATE
packet[3] = currentRoom;
packet[4] = myPid;
packet[5] = 8;         // LEN

pl->x_hi = (spriteX >> 8) & 0xFF;
pl->x_lo = spriteX & 0xFF;
pl->y_hi = (spriteY >> 8) & 0xFF;
```

```
pl->y_lo = spriteY & 0xFF;
pl->frame = currentFrame;
pl->flags = (facingRight ? 0x01 : 0) | (firing ? 0x04 : 0);
pl->data0 = playerHealth;
pl->data1 = 0;

UNAPI_Send(packet, 14);
```


6. SERVIDOR NODE.JS — CÓDIGO BASE

El servidor está implementado en **Node.js puro** usando únicamente el módulo **net** de la librería estándar. Sin dependencias externas. Sin Express. TCP raw. ~150 líneas de código.

6.1 Estructura interna

```
msx-gameserver.js
■
■■■ Protocolo (CMD enum + buildPacket)
■■■ Estado global
■   rooms: Map<roomId, { gameId, players: Map<pid, socket> }>
■   clientState: { auth, roomId, pid, buffer } (por socket)
■
■■■ parsePackets()    ← Buffer acumulativo — crítico para ObsoNET
■■■ handlePacket()    ← Switch de comandos
■■■ broadcastToRoom() ← Relay a todos excepto remitente
■■■ leaveRoom()       ← Limpieza en desconexión
■■■ net.createServer ← Servidor TCP puro
```

6.2 Decisiones de implementación críticas

- **Buffer acumulativo obligatorio:** ObsoNET puede entregar un paquete de 10 bytes en 2–3 chunks TCP distintos según cuándo cae el timer interrupt de InterNestor. El servidor acumula chunks hasta tener el paquete completo.
- **ECONNRESET silenciado:** ObsoNET genera drops frecuentes. Estos errores se loguean solo en debug, no como error crítico.
- **Timeout de 90 segundos:** el Z80 puede tardar en responder. Timeout bajo causa falsos cierres de conexión.
- **Limpieza automática de salas vacías:** cuando el último jugador abandona, la sala se elimina del Map para liberar memoria.
- **Solo P1 puede hacer GAME_START:** el creador de la sala es siempre PID=1 y actúa como host.

6.3 Gestión de GAME_ID

El GAME_ID es un byte opaco para el servidor — solo lo almacena en la sala y lo retorna en ROOM_INFO para que los clientes MSX sepan qué juego se está jugando. Cada juego define su propio ID en el cliente.

GAME_ID	JUEGO (ejemplo)
0x01	Juego 1 (primer juego online)
0x02	Juego 2
0xFF	Reservado / Debug

7. DESPLIEGUE EN VPS UBUNTU

7.1 Instalación del sistema

```
# Ubuntu Server 24.04 LTS – instalación minimal (sin GUI)

sudo apt update && sudo apt upgrade -y
sudo apt install -y ufw fail2ban

# Node.js 20 LTS
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs

node --version    # debe mostrar v20.x.x

# Directorio de la aplicación
sudo mkdir -p /opt/msx-server
sudo cp msx-gameserver.js /opt/msx-server/
cd /opt/msx-server && npm init -y    # sin dependencias externas
```

7.2 Servicio systemd

Archivo: `/etc/systemd/system/msx-server.service`

```
[Unit]
Description=MSX Online Game Server
After=network.target

[Service]
Type=simple
User=nobody
WorkingDirectory=/opt/msx-server
ExecStart=/usr/bin/node msx-gameserver.js
Restart=always
RestartSec=5
MemoryMax=64M

[Install]
WantedBy=multi-user.target

sudo systemctl daemon-reload
sudo systemctl enable msx-server
sudo systemctl start msx-server
sudo systemctl status msx-server
```

7.3 Firewall (ufw)

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow ssh           # Imprescindible – no te cortes del VPS
sudo ufw allow 9876/tcp      # Puerto del servidor MSX
sudo ufw enable
sudo ufw status
```

■ Si los MSX conectan siempre desde IPs conocidas, usar `ufw allow from to any port 9876` en vez de abrirlo al mundo.

7.4 Recursos necesarios en el VPS

RECURSO	MÍNIMO	RECOMENDADO	NOTAS
CPU	1 vCPU	1 vCPU	Node.js es single-thread. 1 core es suficiente.
RAM	256 MB	512 MB	El proceso Node consume ~30–50 MB.
Disco	5 GB	10 GB	Para SO + logs. Sin base de datos.

Red	100 Mbps	100 Mbps	Paquetes pequeños (261 B max). Ancho de banda mínimo.
-----	----------	----------	---

7.5 Monitorización básica

```
# Ver logs en tiempo real
sudo journalctl -u msx-server -f

# Estado del servicio
sudo systemctl status msx-server

# Conexiones activas al puerto MSX
ss -tnp | grep 9876
```

8. RESUMEN DEL STACK Y PRÓXIMOS PASOS

8.1 Stack completo

```
VPS Ubuntu 24.04 minimal
■■■ Node.js 20 LTS (sin dependencias externas)
■■■ msx-gameserver.js (~150 líneas, TCP puro)
■■■ systemd service (autoarranque + restart automático)
■■■ ufw (solo SSH + puerto 9876)

MSX (cliente)
■■■ ObsoNET (hardware Ethernet)
■■■ InterNestor Lite (stack TCP/IP UNAPI 1.1)
■■■ MSXgl / unapi_tcp.h (librería C para Z80)

Protocolo
■■■ TCP raw (sin HTTP, sin WebSocket, sin overhead)
■■■ Header binario 6 bytes + payload máx 255 bytes
■■■ Auth por token 4 bytes (sin TLS — inviable en Z80)
```

8.2 Roadmap sugerido

FASE	TAREA	ESTADO
1	Servidor Node.js base con protocolo definido	Completado (en este doc)
2	Despliegue en VPS + systemd + ufw	Pendiente
3	Cliente MSX: auth + ROOM_CREATE/JOIN en MSXgl	Pendiente
4	Primer juego: STATE_UPDATE con payload 8 bytes	Pendiente
5	Pruebas con hardware ObsoNET real	Pendiente
6	GAME_ID adicionales / más juegos	Futuro
7	Sala de espera / lobby browseable desde MSX	Futuro
8	Estadísticas y ranking por GAME_ID	Futuro

Documento generado como referencia de arquitectura del proyecto MSX Online Game Server. El protocolo y el servidor están diseñados para ser extensibles sin modificaciones al añadir nuevos juegos. Toda la lógica de juego reside en el cliente MSX.